Programowanie obiektowe i inżynieria oprogramowania

Lab 5.1:
- wskaźniki cd,
- konwersja typu zmiennej.

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2024

Argumenty funkcji, referencja, wskaźniki
Tablice

Tablice

main.cpp conversion.h

podstawy_cpp (Globalny za

```cpp
#include <iostream>

using namespace std;

void add1(int tab[], int size)
{
    for (int i = 0; i < size; i++)
    {
        tab[i] += 1;
    }
}

void print(int tab[], int size);

int main()
{
    int tab[] = {1, 2, 3, 4, 5};

    print(tab, 5);
    add1(tab, 5);
    print(tab, 5);
}


void print(int tab[], int size)
{
    for (int i = 0; i < size; i++)
    {
        cout << tab[i];

        if (i + 1 < size)
        {
            cout << ", ";
        }
        else cout << endl;
    }
}
```

Tablice jako argumenty funkcji **<u>zawsze</u> przekazywane są bez ich kopiowania** ze względu na to, że mogą być znacznych wielkości – a ich kopiowanie znacznie spowolniałoby program i pożerało pamięć.

Konsola debugowania programu Microsoft Visual Studio

```
1, 2, 3, 4, 5
2, 3, 4, 5, 6
```

main.cpp conversion.h

podstawy_cpp (Globalny zasięg)

```cpp
#include <iostream>

using namespace std;

void add2(int tab[], int size)
{
    cout << "Adres pierwszego elementu tablicy: " << &tab[0] << endl;

    for (int i = 0; i < size; i++)
    {
        tab[i] += 1;
    }
}

void print(int tab[], int size);

int main()
{
    int tab[] = {1, 2, 3, 4, 5};
    cout << "Adres pierwszego elementu tablicy: " << &tab << endl;
    cout << "Adres pierwszego elementu tablicy: " << &tab[0] << endl;

    print(tab, 5);
    add2(tab, 5);
    print(tab, 5);
}


void print(int tab[], int size)
{
    for (int i = 0; i < size; i++)
    {
        cout << tab[i];

        if (i + 1 < size)
        {
            cout << ", ";
        }
        else cout << endl;
    }
}
```

Dowodzik

Konsola debugowania programu Microsoft Visual Studio

```
Adres pierwszego elementu tablicy: 00000038B78FFC18
Adres pierwszego elementu tablicy: 00000038B78FFC18
1, 2, 3, 4, 5
Adres pierwszego elementu tablicy: 00000038B78FFC18
2, 3, 4, 5, 6
```

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg)

```
1   #include <iostream>
2
3   using namespace std;
4
5   void add2(int* tab, int size)
6   {
7       cout << "Adres pierwszego elementu tablicy: " << tab << endl;
8
9       for (int i = 0; i < size; i++)
10      {
11          *(tab+i) += 1;
12          tab[i] += 1;
13      }
14  }
15
16  void print(int* tab, int size);
17
18  int main()
19  {
20      int tab[] = {1, 2, 3, 4, 5};
21      cout << "Adres pierwszego elementu tablicy: " << &tab << endl;
22      cout << "Adres pierwszego elementu tablicy: " << &tab[0] << endl;
23
24      print(tab, 5);
25      add2(tab, 5);
26      print(&tab[0], 5);
27  }
28
29
30  void print(int* tab, int size)
31  {
32      int* wsk = tab;
33
34      for (int i = 0; i < size; i++)
35      {
36          [redacted]
37
38          cout << tab[i];
39
40          if (i + 1 < size)
41          {
42              cout << ", ";
43          }
44          else cout << endl;
45      }
46  }
47
```

AGH

Różny sposób odwoływania się do tablicy – przez nazwę lub *wskaźnikowo*.

Konsola debugowania programu Microsoft Visual Studio

```
Adres pierwszego elementu tablicy: 000000FBC679F5C8
Adres pierwszego elementu tablicy: 000000FBC679F5C8
1, 2, 3, 4, 5
Adres pierwszego elementu tablicy: 000000FBC679F5C8
3, 4, 5, 6, 7
```

Linia 11: Dzięki temu, że wskaźnik oprócz adresu przechowuje również informacje o typie zmiennej możliwe jest przesuwanie się do kolejnych elementów tablicy!

Argumenty funkcji, referencja, wskaźniki
Wektory

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg)

```cpp
#include <iostream>
#include <vector>

using namespace std;

void print(vector<int> tab);

void add1(vector <int> tab)
{
    int size = tab.size();
    cout << "Adres pierwszego elementu wektora: " << &tab[0] << endl;

    for (int i = 0; i < size; i++)
    {
        tab[i] += 1;
    }

    print(tab);
}

int main()
{
    vector<int> tab = {1, 2, 3, 4, 5};
    cout << "Adres pierwszego elementu wektora: " << &tab[0] << endl;

    print(tab);
    add1(tab);
    print(tab);
}


void print(vector<int> tab)
{
    int size = tab.size();

    for (int i = 0; i < size; i++)
    {
        cout << tab[i];

        if (i + 1 < size)
        {
            cout << ", ";
        }
        else cout << endl;
    }
}
```

Konsola debugowania programu Microsoft Visual Studio

```
Adres pierwszego elementu wektora: 00000258D6548E10
1, 2, 3, 4, 5
Adres pierwszego elementu wektora: 00000258D6548B40
2, 3, 4, 5, 6
1, 2, 3, 4, 5
```

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg) | ad

```cpp
#include <iostream>
#include <vector>

using namespace std;

void print(vector<int> tab);

void add1(vector<int>* tab)
{
    int size = (*tab).size();
    cout << "Adres pierwszego elementu wektora: " << &(*tab)[0] << endl;

    for (int i = 0; i < size; i++)
    {
        (*tab)[i] += 1;
    }

    print(*tab);
}

int main()
{
    vector<int> tab = {1, 2, 3, 4, 5};
    cout << "Adres pierwszego elementu wektora: " << &tab[0] << endl;

    print(tab);
    add1(&tab);
    print(tab);
}


void print(vector<int> tab)
{
    int size = tab.size();

    for (int i = 0; i < size; i++)
    {
        cout << tab[i];

        if (i + 1 < size)
        {
            cout << ", ";
        }
        else cout << endl;
    }
}
```

Konsola debugowania programu Microsoft Visual Studio

```
Adres pierwszego elementu wektora: 00000265713594F0
1, 2, 3, 4, 5
Adres pierwszego elementu wektora: 00000265713594F0
2, 3, 4, 5, 6
2, 3, 4, 5, 6
```

AGH

Argumenty funkcji, referencja, wskaźniki
Klasy

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg)

```cpp
#include <iostream>
#include <vector>

using namespace std;

class TKlasa
{
public:
    int val;

    TKlasa(int init)
    {
        val = init;
    }
};

void add(TKlasa osobnik)
{
    cout << "Adres osobnika: " << &osobnik << endl;
    osobnik.val += 10;
}

int main()
{
    TKlasa os(10);
    cout << "Adres osobnika: " << &os << endl;

    cout << "Wartosc zmiennej osobnika klasy: " << os.val << endl;
    add(os);
    cout << "Wartosc zmiennej osobnika klasy: " << os.val << endl;

}
```

Konsola debugowania programu Microsoft Visual Studio

```
Adres osobnika: 000000E54310F5F4
Wartosc zmiennej osobnika klasy: 10
Adres osobnika: 000000E54310F5D0
Wartosc zmiennej osobnika klasy: 10
```

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg)

```cpp
#include <iostream>
#include <vector>

using namespace std;

class TKlasa
{
public:
    int val;

    TKlasa(int init)
    {
        val = init;
    }
};

void add(TKlasa* osobnik)
{
    cout << "Adres osobnika: " << osobnik << endl;
    (*osobnik).val += 10;
}

int main()
{
    TKlasa os(10);
    cout << "Adres osobnika: " << &os << endl;

    cout << "Wartosc zmiennej osobnika klasy: " << os.val << endl;
    add(&os);
    cout << "Wartosc zmiennej osobnika klasy: " << os.val << endl;

}
```

Konsola debugowania programu Microsoft Visual Studio

```
Adres osobnika: 000000B31E18FAA4
Wartosc zmiennej osobnika klasy: 10
Adres osobnika: 000000B31E18FAA4
Wartosc zmiennej osobnika klasy: 20
```

Konwersja typu zmiennej

main.cpp | conversion.h

podstawy_cpp (Globalny

```cpp
#include <iostream>
#include "conversion.h"

using namespace std;


int main()
{
    int i = 5.2;
    cout << "int i=" << i << endl;

    //int j{ 5.2 };
    //cout << "int j=" << j << endl;
}
```

Konsola debugowania

```
int i=5
```

Lista błędów

Całe rozwiązanie | 0 Błędy | 1 Ostrzeżenie | 0 Komunikaty

Kod	Opis
C4244	"inicjowanie": konwersja z "double" do "int", możliwa utrata danych

main.cpp | conversion.h

podstawy_cpp (Globalny

```cpp
#include <iostream>
#include "conversion.h"

using namespace std;


int main()
{
    int i = 5.2;
    cout << "int i=" << i << endl;

    int j{ 5.2 };
    cout << "int j=" << j << endl;
}
```

Lista błędów

Całe rozwiązanie | 1 Błąd | 1 Ostrzeżenie | 0 Komunikaty

Kod	Opis
C4244	"inicjowanie": konwersja z "double" do "int", możliwa utrata danych
C2397	konwersja z „double" do „int" wymaga konwersji zawężającej

conversion.cpp

podstawy_cpp | (Globalny zasięg) | conv_double_to_int()

```cpp
1  #include "conversion.h"
2  #include <iostream>
3  using namespace std;
4
5
6  void conv_num_to_bool()
7  {
8      int i = 5;
9      double d = 2.5;
10
11     bool bi = i;
12     bool di = d;
13
14     cout << "int i=" << i
15         << "\ndouble d=" << d
16         << "\nbool bi=" << bi
17         << "\nbool di=" << di << "\n\n";
18 }
19
20
21 void conv_int_to_double()
22 {
23     int i = 5;
24     double d = i;
25
26     cout << "int i=" << i << "\ndouble d=" << d << "\n\n";
27 }
28
29 void conv_double_to_int()
30 {
31     double d = 5.8;
32     int i = d;
33     int j = int(d);
34     int k = static_cast<int>(d);
35
36     cout << "double d=" << d << "\nint i=" << i << "\nint j=" << j << "\nint k=" << k << "\n\n";
37 }
```

main.cpp | conversion.h

podstawy_cpp

```cpp
1  #pragma once
2
3
4  void conv_num_to_bool();
5  void conv_int_to_double();
6  void conv_double_to_int();
7
```

Linia 32: konwersja niejawna,
Linia 33: konwersja jawna, stary styl,
Linia 34: konwersja jawna, zalecana

main.cpp conversion.h

podstawy_cpp

```cpp
#include <iostream>
#include "conversion.h"

using namespace std;


int main()
{
    conv_num_to_bool();
    conv_int_to_double();
    conv_double_to_int();
}
```

Konsola debugowania

```
int i=5
double d=2.5
bool bi=1
bool di=1

int i=5
double d=5

double d=5.8
int i=5
int j=5
int k=5
```